

Extreme Value GAMs

Simon Frost

Table of contents

Introduction	1
Setup	2
GEV model	2
Simulate GEV data	2
Fit the GEV model	3
Examine parameter estimates	4
Compare fitted vs true functions	5
GEV fitted vs true plots	5
GEV residual diagnostics	6
GPD model	7
Simulate GPD data	7
Fit the GPD model	8
Examine GPD estimates	8
GPD fitted vs true plots	9
GPD residual diagnostics	10
Model structure	10
Summary	11

Introduction

Extreme value theory (EVT) provides a principled framework for modeling the tails of distributions. Two key models are:

- **Generalized Extreme Value (GEV)** distribution for block maxima: $Y \sim \text{GEV}(\mu, \sigma, \xi)$ with CDF

$$F(y) = \exp \left\{ - \left[1 + \xi \left(\frac{y - \mu}{\sigma} \right) \right]^{-1/\xi} \right\}$$

- **Generalized Pareto Distribution (GPD)** for threshold exceedances: $Y \mid Y > u \sim \text{GPD}(\sigma, \xi)$ with survival function

$$\bar{F}(y) = \left[1 + \xi \left(\frac{y - u}{\sigma} \right) \right]^{-1/\xi}$$

GAM.jl's `evgam` function fits **multi-parameter GAMs** where each distribution parameter can depend on covariates through smooth functions, following the approach of the R [evgam](#) package.

Setup

```
using GAM
using StatsAPI: fitted
using LinearAlgebra: diag
using DataFrames
using CSV
using Random
using Statistics
using Plots
```

GEV model

Simulate GEV data

We generate block maxima where the location and scale vary smoothly with a covariate x :

- Location: $\mu(x) = 5 + 2 \sin(2\pi x)$
- Log-scale: $\log \sigma(x) = -0.5 + 0.5x$, so $\sigma(x) = \exp(-0.5 + 0.5x)$
- Shape: $\xi = 0.1$ (constant, light upper tail)

```
df_gev = CSV.read("data_gev.csv", DataFrame)
n = nrow(df_gev)
x = df_gev.x
y_gev = df_gev.y

mu_true = 5.0 .+ 2.0 .* sin.(2 .* x)
logsigma_true = -0.5 .+ 0.5 .* x
sigma_true = exp.(logsigma_true)
xi_true = 0.1
```

```
println("GEV data: n = $(nrow(df_gev)), y range = [$(round(minimum(y_gev), digits=2)), $(rou
```

GEV data: n = 500, y range = [1.79, 17.11]

Fit the GEV model

We specify one formula per distribution parameter. The GEV has three parameters: location μ , log-scale $\psi = \log \sigma$, and shape ξ .

```
m_gev = evgam(
  @gam_formula(y ~ s(x, k=10, bs=:cr)), # location (x)
  @gam_formula(y ~ s(x, k=8, bs=:cr)), # log-scale (x)
  @gam_formula(y ~ 1)],               # shape (constant)
  df_gev,
  GEVFamily()
)
```

```
MultiParameterModel{GEVFamily}(GEVFamily(), [5.035778002949318, 1.056556449238359, 1.8348704
0.6735083749353833, -1.8764531400150866, -1.8757104827917837, -1.0489110592057713, 0.1366024
0.28479095654890413, 0.0216394109069066, -0.004995967115800963, 0.19215017713130417, 0.07742
0.061140604392183384, 0.006535666609428842, -0.4095447373536006, -0.1665598888035414, -
0.22443062981187722, -0.35321902255802046, -0.10313673306953487, -0.4403807113019337, -
0.19272810192184833, -0.11590091095501875 ... 0.16115630154717564, -0.30750498268852106, -
0.1679229920824913, -0.23268561449091527, -0.3153182996479012, -0.4284784083342434, -
0.11306662125140088, -0.3824188181880798, -0.4280330321528135, -0.6314883763761423], [0.1405
0.12291896228955577 ... 0.8438166805691196 0.0447707000334424; 1.0 -0.10509884017847805 ... 0.70
0.09659396556034533 -0.02903644711468786; 1.0 0.21491894137543824 ... -0.28904064236840654 -
0.08668591615307703], [1.0 -0.14143801811295187 ... 0.700577309129319 0.24661821243013488; 1.0
0.12337120529887255 ... 0.535191584651037 0.42519514492899374; ... ; 1.0 0.7263085982326665 ... -
0.13936080706666798 -0.05448994539787703; 1.0 -0.01898755923276018 ... -0.34094434872632795 -
0.13153300201052215], [1.0; 1.0; ... ; 1.0; 1.0;;]], Vector{ConstructedSmooth}[[ConstructedSmoo
0.00020556773079296223 ... 0.00023744133032928093 -0.00045890992055891476; -
0.00020556773079296223 0.009556289054786445 ... -0.0012946855016136305 -3.75008465368012e-
5; ... ; 0.00023744133032928093 -0.0012946855016136305 ... 0.005699586717268395 -
5.155389881705624e-5; -0.00045890992055891476 -3.75008465368012e-5 ... -5.155389881705624e-
5 0.0012938820382817522], [0.0015475098674477053 -0.00020556773079296223 ... 0.0002374413303292
0.00045890992055891476; -0.00020556773079296223 0.009556289054786445 ... -
0.0012946855016136305 -3.75008465368012e-5; ... ; 0.00023744133032928093 -
0.0012946855016136305 ... 0.005699586717268395 -5.155389881705624e-5; -0.00045890992055891476 -
3.75008465368012e-5 ... -5.155389881705624e-5 0.0012938820382817522], 687.464980961885, 724.93
```

Examine parameter estimates

```
println("Number of parameters: ", nparams(m_gev))
println("Converged: ", m_gev.converged)
println("Negative log-likelihood: ", round(m_gev.nll, digits=2))
println("REML score: ", round(m_gev.reml, digits=2))
```

Number of parameters: 3
Converged: true
Negative log-likelihood: 687.46
REML score: 724.93

Location parameter coefficients and fitted values:

```
mu_coefs = param_coef(m_gev, 1)
mu_hat = param_eta(m_gev, 1)
println("Location coefficients (first 5): ", round.(mu_coefs[1:min(5, length(mu_coefs))], digits=2))
println("Location fitted range: [$(round(minimum(mu_hat), digits=2)), $(round(maximum(mu_hat), digits=2))]")
println("Location true range: [$(round(minimum(mu_true), digits=2)), $(round(maximum(mu_true), digits=2))]")
```

Location coefficients (first 5): [5.036, 1.057, 1.835, 1.768, 0.738]
Location fitted range: [3.02, 7.03]
Location true range: [3.0, 7.0]

Log-scale parameter:

```
psi_coefs = param_coef(m_gev, 2)
psi_hat = param_eta(m_gev, 2)
println("Log-scale coefficients (first 5): ", round.(psi_coefs[1:min(5, length(psi_coefs))], digits=2))
println("Log-scale fitted range: [$(round(minimum(psi_hat), digits=2)), $(round(maximum(psi_hat), digits=2))]")
println("Log-scale true range: [$(round(minimum(logsigma_true), digits=2)), $(round(maximum(logsigma_true), digits=2))]")
```

Log-scale coefficients (first 5): [-0.285, 0.022, -0.005, 0.192, 0.077]
Log-scale fitted range: [-0.72, 0.23]
Log-scale true range: [-0.5, -0.0]

Shape parameter (constant):

```
xi_coefs = param_coef(m_gev, 3)
println("Shape coefficient: ", round(xi_coefs[1], digits=4))
println("True shape: ", xi_true)
```

```
Shape coefficient: 0.1406
True shape: 0.1
```

Compare fitted vs true functions

```
ord = sortperm(x)
x_sorted = x[ord]

cor_mu = cor(mu_hat, mu_true)
cor_psi = cor(psi_hat, logsigma_true)
println("Correlation (fitted vs true location): ", round(cor_mu, digits=4))
println("Correlation (fitted vs true log-scale): ", round(cor_psi, digits=4))
println("RMSE (location): ", round(sqrt(mean((mu_hat .- mu_true).^2)), digits=3))
println("RMSE (log-scale): ", round(sqrt(mean((psi_hat .- logsigma_true).^2)), digits=3))
```

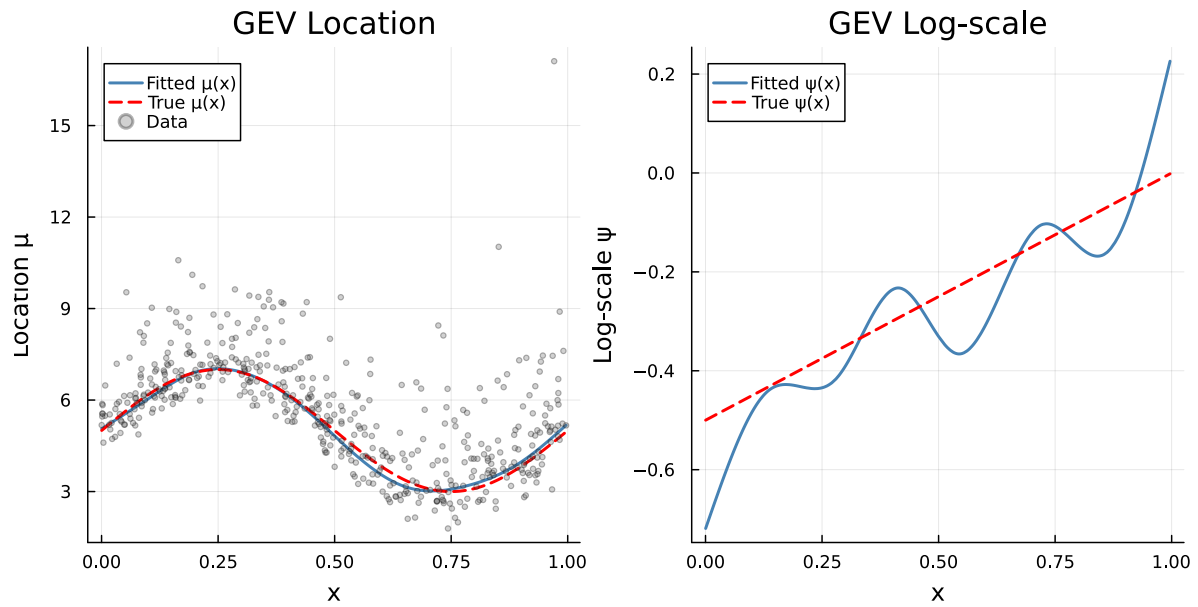
```
Correlation (fitted vs true location): 0.9952
Correlation (fitted vs true log-scale): 0.9293
RMSE (location): 0.138
RMSE (log-scale): 0.078
```

GEV fitted vs true plots

```
p1 = plot(x_sorted, mu_hat[ord], label="Fitted (x)", color=:steelblue, lw=2,
          xlabel="x", ylabel="Location ", title="GEV Location")
plot!(p1, x_sorted, mu_true[ord], label="True (x)", color=:red, ls=:dash, lw=2)
scatter!(p1, x, y_gev, label="Data", color=:grey40, markersize=2, alpha=0.3)

p2 = plot(x_sorted, psi_hat[ord], label="Fitted (x)", color=:steelblue, lw=2,
          xlabel="x", ylabel="Log-scale ", title="GEV Log-scale")
plot!(p2, x_sorted, logsigma_true[ord], label="True (x)", color=:red, ls=:dash, lw=2)

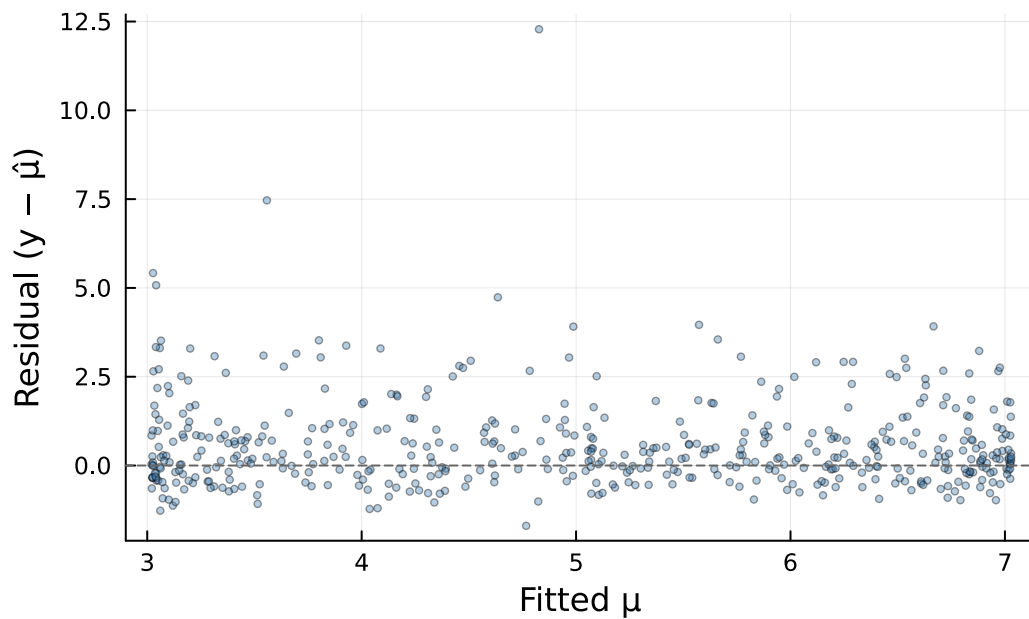
plot(p1, p2; layout=(1, 2), size=(800, 400))
```



GEV residual diagnostics

```
resid_gev = y_gev .- mu_hat
scatter(mu_hat, resid_gev, xlabel="Fitted ", ylabel="Residual (y - ^)",
        title="GEV Residuals vs Fitted", color=:steelblue, markersize=2, alpha=0.4,
        legend=false)
hline!([0.0], color=:grey40, ls=:dash)
```

GEV Residuals vs Fitted



GPD model

Simulate GPD data

For threshold exceedances, we simulate GPD data with covariate-dependent scale:

- Log-scale: $\log \sigma(x) = 0.5 \sin(2\pi x)$
- Shape: $\xi = 0.15$ (constant)

```
df_gpd = CSV.read("data_gpd.csv", DataFrame)
n_gpd = nrow(df_gpd)
x_gpd = df_gpd.x
y_gpd = df_gpd.y

logsigma_gpd_true = 0.5 .* sin.(2 .* x_gpd)
sigma_gpd_true = exp.(logsigma_gpd_true)
xi_gpd_true = 0.15

println("GPD data: n = $(nrow(df_gpd)), y range = [$(round(minimum(y_gpd), digits=2)), $(roun
```

GPD data: n = 500, y range = [0.0, 12.35]

Fit the GPD model

The GPD has two parameters: log-scale $\psi = \log \sigma$ and shape ξ .

```
m_gpd = evgam(  
  @gam_formula(y ~ s(x, k=10, bs=:cr)), # log-scale (x)  
  @gam_formula(y ~ 1)], # shape (constant)  
  df_gpd,  
  GPDFamily()  
)
```

```
MultiParameterModel{GPDFamily}(GPDFamily(0.0), [0.5450726792017321, 0.01308663260846314, -  
0.08368522714542513, -0.1823945581407308, -0.23534058426636836, -0.20832192204361002, -  
0.11654555431262244, 0.013229783990656536, 0.16707406759524607, 0.3333888201061366, 0.062471  
0.12422550358987867 ... -0.12041743435729216 -0.04021065844661172; 1.0 -0.13042251125561036 ...  
0.12187198223679756 ... -0.12133151977556947 -0.03914430141811146; 1.0 -0.10861130400117933 ...  
0.11520961231045118 -0.042781265610324345], [1.0; 1.0; ... ; 1.0; 1.0;;]], Vector{ConstructedS  
0.0003691846762570065 -0.002251355396011936; 0.00030750181318067824 0.0026879772317192577 ...  
0.0017151679667873735 -0.00033996961358779746; ... ; -0.0003691846762570065 -  
0.0017151679667873735 ... 0.01928444755219919 0.0003168644531699814; -0.002251355396011936 -  
0.00033996961358779746 ... 0.0003168644531699814 0.0023876892360554175], [0.00437409145597252  
0.0003691846762570065 -0.002251355396011936; 0.00030750181318067824 0.0026879772317192577 ...  
0.0017151679667873735 -0.00033996961358779746; ... ; -0.0003691846762570065 -  
0.0017151679667873735 ... 0.01928444755219919 0.0003168644531699814; -0.002251355396011936 -  
0.00033996961358779746 ... 0.0003168644531699814 0.0023876892360554175], 803.7722049808459, 81
```

Examine GPD estimates

```
println("Converged: ", m_gpd.converged)  
println("Negative log-likelihood: ", round(m_gpd.nll, digits=2))  
  
psi_gpd_hat = param_eta(m_gpd, 1)  
xi_gpd_hat = param_coef(m_gpd, 2)  
  
println("Log-scale fitted range: [$(round(minimum(psi_gpd_hat), digits=2)), $(round(maximum(psi_gpd_hat), digits=2))]  
println("Log-scale true range: [$(round(minimum(logsigma_gpd_true), digits=2)), $(round(maximum(logsigma_gpd_true), digits=2))]  
println("Shape estimate: ", round(xi_gpd_hat[1], digits=4))  
println("True shape: ", xi_gpd_true)  
  
cor_gpd = cor(psi_gpd_hat, logsigma_gpd_true)  
println("Correlation (fitted vs true log-scale): ", round(cor_gpd, digits=4))
```

Converged: true
 Negative log-likelihood: 803.77
 Log-scale fitted range: [0.36, 0.9]
 Log-scale true range: [-0.5, 0.5]
 Shape estimate: 0.0625
 True shape: 0.15
 Correlation (fitted vs true log-scale): -0.2538

GPD fitted vs true plots

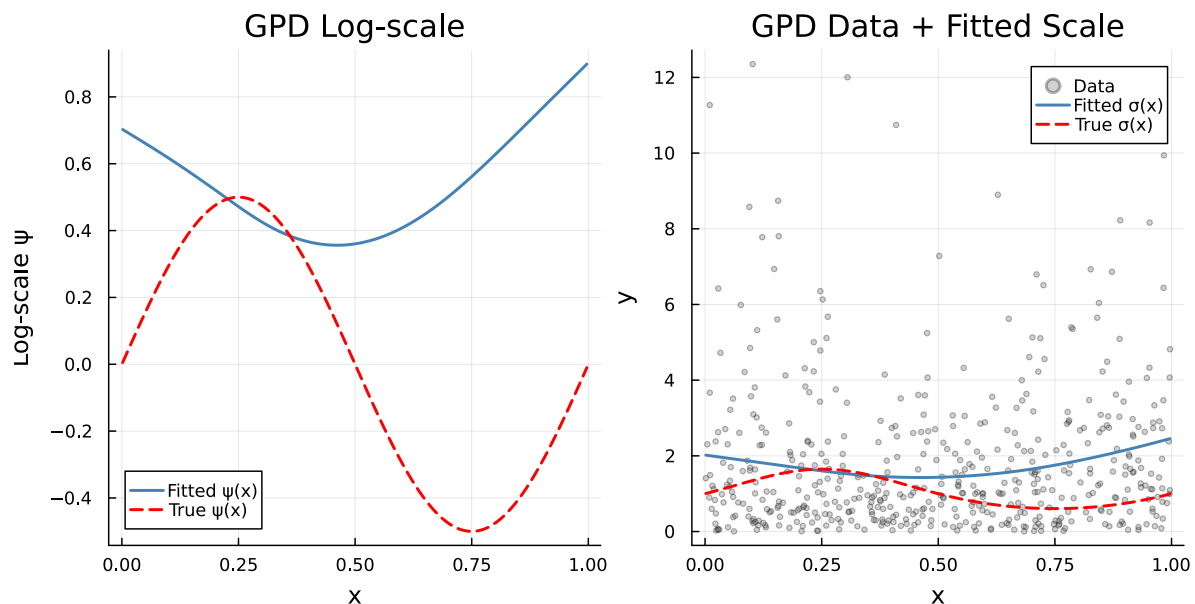
```

ord_gpd = sortperm(x_gpd)
x_gpd_sorted = x_gpd[ord_gpd]

p3 = plot(x_gpd_sorted, psi_gpd_hat[ord_gpd], label="Fitted (x)", color=:steelblue, lw=2,
          xlabel="x", ylabel="Log-scale ", title="GPD Log-scale")
plot!(p3, x_gpd_sorted, logsigma_gpd_true[ord_gpd], label="True (x)", color=:red, ls=:dash,

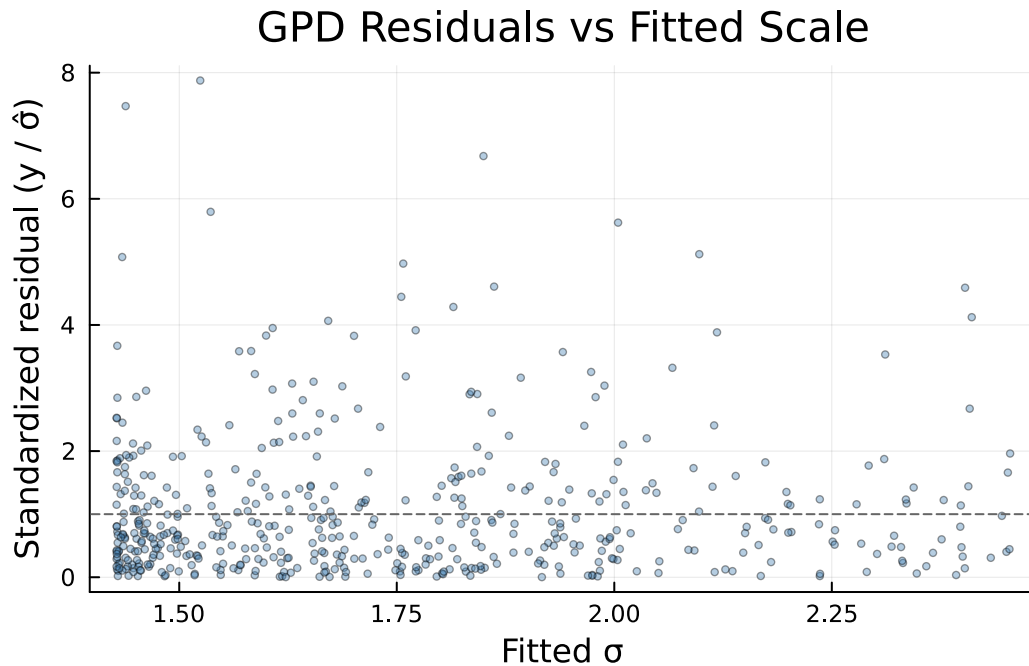
p4 = scatter(x_gpd, y_gpd, label="Data", color=:grey40, markersize=2, alpha=0.3,
             xlabel="x", ylabel="y", title="GPD Data + Fitted Scale")
plot!(p4, x_gpd_sorted, exp.(psi_gpd_hat[ord_gpd]), label="Fitted (x)", color=:steelblue, lw=
plot!(p4, x_gpd_sorted, sigma_gpd_true[ord_gpd], label="True (x)", color=:red, ls=:dash, lw=

plot(p3, p4; layout=(1, 2), size=(800, 400))
  
```



GPD residual diagnostics

```
sigma_gpd_hat = exp.(psi_gpd_hat)
resid_gpd = y_gpd ./ sigma_gpd_hat
scatter(sigma_gpd_hat, resid_gpd, xlabel="Fitted ", ylabel="Standardized residual (y / ^)",
        title="GPD Residuals vs Fitted Scale", color=:steelblue, markersize=2, alpha=0.4,
        legend=false)
hline!([1.0], color=:grey40, ls=:dash)
```



Model structure

The MultiParameterModel returned by `evgam` stores:

```
println("Type: ", typeof(m_gev))
println("Total coefficients: ", length(m_gev.coefficients))
println("EDFs: ", round.(m_gev.edf, digits=2))
println("Log smoothing parameters: ", round.(m_gev.sp, digits=2))
println("Covariance matrix size: ", size(m_gev.Vp))
```

Type: MultiParameterModel{GEVFamily}

Total coefficients: 19
EDFs: [1.0, 0.81, 0.9, 0.87, 0.82, 0.85, 0.84, 0.86, 0.85, 0.75, 1.0, 0.09, 0.01, 0.0, 0.06,
Log smoothing parameters: [3.14, 13.19]
Covariance matrix size: (19, 19)

Standard errors via the posterior covariance:

```
se_all = sqrt.(diag(m_gev.Vp))  
println("Standard errors (first 5): ", round.(se_all[1:min(5, length(se_all))], digits=4))
```

Standard errors (first 5): [0.0393, 0.0978, 0.0854, 0.0912, 0.0974]

Summary

GAM.jl's `evgam` provides:

Feature	Description
<code>GEVFamily()</code>	GEV distribution (3 parameters: , log ,)
<code>GPDFamily()</code>	GPD distribution (2 parameters: log ,)
Multi-formula interface	One formula per distribution parameter
REML smoothing	Automatic smoothing parameter estimation
<code>param_coef(m, k)</code>	Coefficients for parameter k
<code>param_eta(m, k)</code>	Fitted linear predictor for parameter k